

Table des matières

1 Définition	1
1.1 Fonction booléenne	1
1.2 Forme normal algébrique et degré	2
1.3 Code de Reed-Muller	2
1.4 Non linéarité	2
1.5 Rayon de recouvrement	2
2 Problématique	2
2.1 Inégalité de ρ	3
2.1.1 Décomposition de Plotkin	3
3 calcul de distance	3
3.1 énumération	3
4 Travail sur les bent	4
5 Algo	4
5.1 Algo probabiliste	4
5.2 Algo Tarvernier like	4

1 Définition

1.1 Fonction booléenne

Une fonction booléenne en m variables est une fonction qui prend en paramètre un m -uplet binaire et qui a ses valeurs dans $\{0, 1\}$.

$$f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$$

on note $B(m)$ l'ensemble des fonctions booléennes en m variables. Soit $f \in B(m)$, on définit le poids de f comme le nombre de fois où la fonction prend la valeur 1.

$$\text{wt}(f) = \#\{f(x) = 1 \mid x \in \mathbb{F}_2^m\}$$

On peut représenter une fonction par la table de ses valeurs :

$$\text{TT}(f) := [f(0), f(1), \dots, f(2^m - 1)]$$

La distance de Hamming de deux fonctions booléennes f et g est définie par le nombre de fois où les fonctions ne partagent pas de valeur égale.

$$d_H(f, g) = \#\{f(x) \neq g(x) \mid x \in \mathbb{F}_2^m\} = \text{wt}(f + g)$$

Soit C un code dont ses éléments sont un sous ensemble des fonctions booléennes en m variables et f une fonction booléenne en m variables, on définit la distance de f au code comme la distance de Hamming minimal entre f et tous les mots de C .

$$d(f, C) = \min_{c \in C} d_H(f, c)$$

1.2 Forme normal algébrique et degré

Toute fonction booléenne peut s'écrire comme une somme de monômes nommé ANF (algebraic normal form)

$$f(x) = \sum_{I \subseteq \{1, \dots, m\}} a_I \prod_{i \in I} x_i$$

Le degré d'une fonction est le degré maximal de ses monômes. La valuation d'une fonction et le degré minimal de ses monômes. On note $B_s^t(m)$ l'ensemble des fonctions en m variables de valuation au moins s et de degré au plus t .

$$B_s^t(m) := \{f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2 \mid \text{val}(f) \geq s \quad \deg(f) \leq t\}$$

1.3 Code de Reed-Muller

Le code de Reed-Muller en m variables et d'ordre k $\text{RM}(k, m)$ est l'ensemble des fonctions booléennes de degré inférieur ou égale à k .

$$\text{RM}(k, m) := \{f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2 \mid \deg(f) \leq k\}$$

1.4 Non linéarité

Soit un code $\text{RM}(k, m)$ et une fonction $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ on définit la non linéarité d'ordre k de f comme sa distance au code autrement dit à sa distance de toutes les fonctions de degré au plus k .

$$\text{NL}_k(f) = d(f, \text{RM}(k, m))$$

1.5 Rayon de recouvrement

$\rho(k, m)$ Le rayon de recouvrement d'un code $\text{RM}(k, m)$ de Reed-Muller est la distance maximale entre ce dernier et l'ensemble des fonctions booléennes en m variables.

$$\rho(k, m) = \max_{f \in B(m)} \text{NL}_k(f) = \max_{f \in B(m)} d(f, \text{RM}(k, m))$$

1

2 Problématique

Connaitre le rayon de recouvrement d'un code de Reed Muller est interessant pour "blablablablablabla". Mais la taille de l'espace des fonctions booléennes est immense. Pour $m = 8$ il existe 2^{2^8} soit 2^{256} fonctions booléennes. De plus si on veut calculer la non linéarité d'ordre 2 pour toutes les fonctions, calculer la distance entre une fonction de $B(8)$ et tout $\text{RM}(2, 8)$ n'est pas non plus trivial car ce code comporte 2^{37} éléments. Ce tableau (voir tab 1) recense toutes les valeurs de rho connu jusqu'à $m = 10$. On peut voir que l'on connait toutes les valeurs de rho jusqu'à $m = 7$. Au-delà certaine valeur sont compris dans un interval. Nottament on s'intéresse à $88 \leq \rho(2, 8) \leq 96$ et $56 \leq \rho(3, 8) \leq 60$

TABLE 1 – Tableau des valeur de $\rho(k, n)$

$k \backslash n$	3	4	5	6	7	8	9	10
1	2	6	12	28	56	120	242-244	496
2	1	2	6	18	40	88-96	196-216	400-460
3	0	1	2	8	20	56-60	111-156	194-372
4		0	1	2	8	26	58-86	≤ 242
5			0	1	2	10	28-36	≤ 122
6				0	1	2	10	≤ 46
7					0	1	2	12
8						0	1	2

2.1 Inégalité de ρ

On sait que

$$\rho(k, n) \leq \rho(k, n-1) + \rho(k-1, n-1)$$

donc pour $\text{RM}(2, 8)$ on voudrait savoir si il existe une fonction f qui atteindrait cette borne supérieure tel que $\text{NL}_2(f) = 96$.

2.1.1 Décomposition de Plotkin

Toute fonction booléenne f en m variable de degré k autrement dit $f \in \text{RM}(k, m)$ peut s'écrire sous la forme

$$f = x_m g + h$$

avec $g \in \text{RM}(k-1, m-1)$ et $h \in \text{RM}(k, m-1)$ si $x_m = 0, f = h$ et si $x_m = 1, f = g + h$ la table de vérité de toute fonction est donc de la forme

$$[h \mid h + g]$$

Ce qui veut dire que pour que $\text{NL}_2(f) = 96$ il faut que pour n'importe quelle fonction $t = x_8 v + u \in \text{RM}(2, 8)$

$$\text{wt}(t + f) \geq 96 \equiv \text{wt}(h + v) + \text{wt}(h + v + g + u) \geq 96$$

3 calcul de distance

3.1 énumération

Dans ce paragraphe je vais présenter une methode pour calculer la distance d'une fonction f à un code C . la manière la plus simple est consiste à énumérer tous les mots du code. Pour cela il faut générer toutes les combinaisons linéaires des lignes de la matrice génératrice G du code. La matrice G comporte k ligne $l_1, l_2, \dots, l_k$ donc chaque mot c du code s'écrit

$$c = \sum_{i=1}^k u_i l_i \quad \text{où} \quad u \in \mathbb{F}_2^k \quad u = (u_1, u_2, \dots, u_k)$$

Nous avons donc besoin d'énumérer tous les u mais au lieu d'utiliser l'ordre binaire classique $0 = (000), 1 = (001), 2 = (010) \dots$ on utilise le code de Gray afin de changer un bit à la fois, l'indice de ce bit correspond à l'indice de la ligne à additionner afin de calculer le prochain mot du code. Par exemple pour $k = 3$:

$$G = \begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix}$$

u	000	001	011	010	110	111	101	100
ligne additionné	0	l_1	l_2	l_1	l_3	l_1	l_2	l_1
c	0	l_1	$l_1 + l_2$	l_2	$l_3 + l_2$	$l_3 + l_2 + l_1$	$l_3 + l_1$	l_3

Dans l'exemple pour passer du mot $l_1 + l_2$ représenté par le vecteur $u = 011$ on passe à la prochaine valeur de u dans le code de Gray qui est $u = 010$, le bit d'indice 1 a été modifié donc pour passer au prochain mot il suffit de faire $l_1 + l_2 + l_1 = l_2$.

4 Travail sur les bent

Une fonction booléenne f est dite bent si sa distance aux fonctions affines est la plus grande possible $NL_1(f) = \rho(1, m)$ nous avons voulu étudier la non linéarité d'ordre 2 pour les bent en 8 variables. Calculer la non linéarité d'ordre 2 de toutes les fonctions bent aurait été impossible étant donné qu'il y en a 190 millions et 2^{37} mots dans $RM(2, 8)$. Il nous fallait donc trouver un moyen d'estimer rapidement la non linéarité d'une fonction à l'aide d'un algorithme probabiliste. La classe latérale d'une fonction f modulo C est l'ensemble des translatés de f par les éléments de C

$$f + C = \{f + c \mid c \in C\}$$

Calculer la distance de f à un code revient donc à trouver le poids minimal de sa classe latérale. L'algorithme probabiliste que l'on a utilisé a pour but de créer un translaté aléatoire de f de poids au plus $m - k$ avec m et k la longueur et la dimension du code. Cette algorithme a été très rapide et efficace pour trouver et éliminer les fonctions de faible non linéarité ce qui nous a permis de traiter l'intégralité des fonctions bent en quelques heures. Nous avons trouvé que la non linéarité d'ordre 2 des fonctions bent est de 88. On pourrait dire par intuition que $\rho(2, 8) = 88$ mais rien n'est sûr.

5 Algo

5.1 Algo probabiliste

voir Algo 1

5.2 Algo Tarvernier like

Voir Algo 2

Algorithm 1 Random translate

Require: G : Une matrice génératrice d'un code $\text{RM}(k, m)$ de dimension $[K, M]$
 f : Une fonction booléenne en n variable représentée par sa table de vérité de longueur N

```
1: for  $i \leftarrow 0$  to  $K - 1$  do
2:   repeat
3:      $pivot \leftarrow \text{rand}() \bmod M$ 
4:   until  $G[i][pivot] = 1$ 
5:   for  $j \leftarrow i$  to  $K - 1$  do
6:     if  $j \neq i$  and  $G[j][pivot] = 1$  then
7:        $G[j] \leftarrow G[j] + G[i]$ 
8:     end if
9:   end for
10:  if  $f[i] = 1$  then
11:     $f \leftarrow f + G[i]$ 
12:  end if
13: end for
14: return  $f$ 
```

Algorithm 2 Calcule distance

Require: Une fonction booléenne en n variable : $f \in \mathcal{B}(m) = x_n g + h$

L'ordre de non linéarité recherché : k

M et K la longueur et la dimension du code $\text{RM}(k, m)$

```
1:  $best \leftarrow M$ 
2: for  $u \in \text{RM}(k - 1, m - 1)$  do
3:   if  $\text{wt}(g + u) < best$  then
4:     for  $v \in \text{RM}(k, m - 1,)$  do
5:        $best \leftarrow \min \text{wt}(h + v) + \text{wt}(h + v + g + u)$ 
6:     end for
7:   end if
8: end for
9: return  $best$ 
```
